

Step■by■Step Implementation Guide: Firestore Native Vector Search RAG Application

Purpose

This document provides step■by■step instructions to build a low■cost Retrieval■Augmented Generation (RAG) application using React, Firebase, Firestore Native Vector Search, and Google's embedding models. The system allows users to ask questions and receive answers grounded in your uploaded documents.

Phase 1: User Interface Setup

1. Create a frontend application using React and Vite.
2. Design a chat interface where users can type questions.
3. Add input field, send button, and response display area.
4. Connect the frontend to Firebase Functions.
5. Ensure the UI can send user questions securely to the backend.
6. Deploy the frontend to Firebase Hosting.

Phase 2: Firebase and Firestore Setup

1. Create a Firebase project.
2. Enable Firestore Database.
3. Enable Firebase Functions.
4. Configure Firebase Hosting.
5. Enable Firestore Native Vector Search indexing.
6. Configure Firestore security rules.
7. Create Firestore collections for documents and chunks.

Phase 3: Document Storage Setup

1. Store original documents locally or in Google Cloud Storage.
2. Ensure documents are in supported formats such as PDF or text.
3. Organize documents into logical categories.
4. Prepare documents for ingestion processing.

Phase 4: Document Processing and Embedding

1. Extract text from documents.
2. Split text into smaller chunks.
3. Generate vector embeddings for each chunk.
4. Store chunk text and embedding vectors in Firestore.
5. Store document metadata in Firestore.
6. Verify embeddings are correctly stored.

Phase 5: Firestore Vector Search Configuration

1. Enable vector indexing in Firestore.
2. Configure vector dimension to match embedding model.
3. Verify Firestore can perform nearest-neighbor searches.
4. Optimize Firestore indexing for performance.

Phase 6: Backend Query System

1. Receive user question from frontend.
2. Convert question into vector embedding.
3. Perform Firestore nearest-neighbor search.
4. Retrieve most relevant document chunks.
5. Send retrieved content to AI model.
6. Generate grounded response.
7. Return answer to frontend.

Phase 7: AI Response Generation

1. Use embedding model to understand query.
2. Provide relevant chunks as context.
3. Generate answer using Gemini model.
4. Return answer and references.
5. Display answer in chat interface.

Phase 8: Deployment

1. Deploy Firebase Functions.
2. Deploy Firebase Hosting.
3. Verify full system functionality.
4. Monitor performance and cost.
5. Optimize for speed and efficiency.

Required Libraries and Dependencies

Frontend Dependencies:

- React
- Vite
- Firebase SDK
- TypeScript
- Tailwind CSS (optional)

Backend Dependencies:

- Firebase Functions
- Firebase Admin SDK
- Google Gen AI SDK
- Node.js

Python Dependencies:

- firebase-admin
- google-generativeai
- pdf processing library (such as pypdf or pdfplumber)

Cloud Services:

- Firebase
- Firestore Database
- Google Cloud Storage (optional)
- Vertex AI Embeddings

System Workflow Summary

1. User submits question.
2. Backend generates embedding.
3. Firestore finds nearest matching vectors.
4. AI generates answer from retrieved data.
5. Answer displayed to user.

End of Guide